

OPPOSING-ARGUMENT SIMULATOR FOR SELF-REPRESENTED LITIGANTS

Jurisdiction-Grounded Retrieval-Augmented Generation (RAG)
for Access-to-Justice Technology

Software Requirements Specification & Technical Architecture

Prepared by:

Hasana Zahid

Bachelor of Science in Artificial Intelligence

Registration No: SP24-BAI-060

COMSATS University Islamabad, Islamabad Campus

Date: July 2026

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW
2. PROBLEM STATEMENT & OBJECTIVES
3. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)
4. RETRIEVAL-AUGMENTED GENERATION (RAG) ARCHITECTURE
5. GENERATIVE AI & ADVERSARIAL PROMPTING STRATEGY
6. VECTOR DATABASE & EMBEDDING STRATEGY
7. IMPLEMENTATION ROADMAP (4 MODULAR STEPS)
8. DEPLOYMENT & PRODUCTION ARCHITECTURE
9. EVALUATION METRICS & QUALITY ASSURANCE
10. SAFETY FRAMEWORKS & LEGAL SAFEGUARDS
11. TECHNICAL SPECIFICATION & CODE PATTERNS
12. APPENDICES

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

The **Opposing-Argument Simulator** is a generative AI application designed to provide self-represented litigants with realistic adversarial case preparation through jurisdiction-grounded retrieval-augmented generation (RAG). By integrating domain-specific legal knowledge bases with advanced language models, this system simulates the counterarguments, procedural objections, and counterevidence an opposing party would likely present—enabling laypeople to rehearse their cases before hearing and dramatically improving preparedness and equity in the civil justice system.

Core Innovation:

Traditional legal-tech tools focus on one-directional support (document assembly, procedure explanation, case law search). This system is the first to combine RAG-grounded retrieval with adversarial role-play prompting, generating the strongest possible counterarguments a self-represented litigant must anticipate. Every citation is verified against retrieved legal sources, ensuring hallucination-free output suitable for real court preparation.

Key Deliverables:

- Jurisdiction-grounded RAG pipeline using Harvard Caselaw Access Project (CAP) dataset
- Adversarial LLM engine generating defensible counter-arguments with strict citation verification
- Serverless-optimized web application with SSE streaming to bypass timeout constraints
- Interactive case rehearsal interface with rebuttal preparation workspace
- Evaluation framework measuring citation accuracy, response latency, and user preparedness

2. PROBLEM STATEMENT & OBJECTIVES

The Justice Access Gap

Millions of self-represented litigants appear in U.S. civil courts annually—primarily in small claims, tenancy, and family law matters. These individuals must simultaneously absorb complex procedural rules, substantive legal standards, and evidence considerations, while entering hearings *blind to the specific counterarguments and precedents* opposing counsel intends to raise. This structural asymmetry of preparation—not legal merit—is a well-documented driver of adverse outcomes.

Why Existing Solutions Fall Short

- No existing tool simulates the opposing party's likely arguments, procedural defenses, or counterevidence
- General-purpose LLMs hallucinate statutes and cite non-existent case law when used without jurisdictional grounding
- Laypeople cannot assess the strength of their position relative to the adversary's expected response
- Legal tech is heavily skewed toward document assembly rather than case preparation and rehearsal

Project Objectives

O1: Case Fact Structuring — Convert unstructured litigant narratives into queryable case schemas (parties, claims, jurisdiction, dates, evidence)

O2: Jurisdiction-Grounded Retrieval — Build a RAG pipeline ensuring every argument is traceable to verified statutory or case-law sources from the target jurisdiction

O3: Adversarial Simulation — Generate realistic counter-arguments, procedural objections, and counterevidence grounded strictly in retrieved legal authority

O4: Rebuttal Preparation & Safety — Enable users to draft counter-rebuttals while maintaining persistent disclaimers that the tool provides practice, not legal counsel

3. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 System Scope & Boundaries

The system is strictly limited to automated case-preparation practice and tactical analysis. It does NOT generate signed court documents, provide non-grounded legal advice, or replace attorney representation. Processing is confined to four operational pillars: Case Intake, Jurisdiction-Grounded Retrieval, Adversarial Simulation, and Rebuttal Preparation.

3.2 Functional Requirements (Four Pillars)

Pillar 1: Case Intake & Fact Structuring

FR-1.1: Interactive form-guided wizard interface for free-text case narrative entry

FR-1.2: NLP-based extraction layer producing deterministic JSON schema: {Jurisdiction, ClaimType, KeyDates, DisputedFacts, AvailableEvidence}

FR-1.3: Validation layer flagging missing context (e.g., dates for statute-of-limitations checks)

FR-1.4: Explicit jurisdiction selector ensuring downstream queries target correct legal database

Pillar 2: Jurisdiction-Grounded Retrieval

FR-2.1: Vector-similarity search over jurisdiction-specific legal corpus (Harvard CAP dataset)

FR-2.2: Document segmentation by court opinion (not arbitrary text chunks)

FR-2.3: Embedding generation via lightweight encoder (e.g., all-MiniLM-L6-v2); storage in Chroma/Qdrant

FR-2.4: Top-k retrieval (k=5-10) ranked by cosine similarity; return full citation + relevant text excerpts

Pillar 3: Opposing-Argument Simulation Engine

FR-3.1: Adversarial LLM prompt engineering simulating opposing counsel behavior

FR-3.2: Output highlighting expected counter-arguments, procedural defenses, counterevidence vulnerabilities

FR-3.3: Strict citation linking: every claim must map to a retrieved case/statute via deterministic regex verification

FR-3.4: Confidence scoring for each counter-argument based on retrieved source strength

Pillar 4: Rebuttal Preparation & Safety Framing

FR-4.1: Split-screen interface: simulated opposing argument (left) paired with user rebuttal text box (right)

FR-4.2: Non-dismissible disclaimer overlay: 'This tool provides educational practice. It is not legal advice and does not replace qualified attorney representation.'

FR-4.3: Full-text export to PDF: 'Hearing Rehearsal Guide' combining case facts, simulated challenges, and user counter-arguments

3.3 Non-Functional Requirements (NFRs)

NFR-1: Hallucination Mitigation: Every citation must have 1:1 verification against retrieved source chunks. Generative completions without matching evidence are programmatically discarded. Target: $G_v = C_v/C_t = 1.00$ (100% verification rate).

NFR-2: Response Latency: To manage Vercel serverless timeouts (15s limit on hobby tier), all generative responses stream via Serverless Event Streaming (SSE). Token-block rendering <3.0s from user query.

NFR-3: Data Privacy & Local Storage: Case records and litigant input remain in browser session or transient memory. No persistent server storage of civil litigation details. Compliance with data minimization principles.

NFR-4: System Availability: Target 99.5% uptime during business hours. Graceful degradation if vector database becomes unavailable (fallback to mock responses with disclaimers).

NFR-5: Scalability: Architecture supports concurrent case intakes without performance degradation. Vector database queries scale linearly with corpus size (<200ms per query for 100k+ cases).

NFR-6: Accessibility: WCAG 2.1 AA compliance. Readable font sizes, high-contrast color schemes, keyboard navigation, screen-reader support.

4. RETRIEVAL-AUGMENTED GENERATION (RAG) ARCHITECTURE

4.1 RAG System Overview

The RAG architecture is the cornerstone of hallucination-free legal simulation. Rather than relying on the language model's pre-trained knowledge (which may be outdated, unverifiable, or jurisdictionally irrelevant), RAG retrieves relevant legal authorities *before* generation. This transforms the task from 'generate legal arguments' to 'synthesize retrieved legal authorities into persuasive arguments.'

4.2 RAG Pipeline Stages

Stage 1: Document Ingestion & Normalization

The Harvard Caselaw Access Project (CAP) dataset is streamed from Hugging Face and segmented by court opinion. Each opinion is parsed to extract case metadata (case name, date, jurisdiction, court level, headnotes) and opinion text. Documents are deduplicated and validated.

Stage 2: Chunking & Contextualization

Opinions are chunked by logical sections (e.g., Holding, Reasoning, Dissent) rather than arbitrary tokens. Each chunk retains citation metadata and source reference. Chunk size: 300-500 tokens, with 50-token overlap to preserve context.

Stage 3: Embedding Generation

All chunks are encoded using a lightweight, domain-optimized embedding model (e.g., sentence-transformers all-MiniLM-L6-v2 or legal-BERT). Embeddings are 384-dimensional vectors suitable for efficient similarity search. Embedding generation is parallelized to handle 100k+ documents.

Stage 4: Vector Database Storage

Embeddings are stored in Qdrant or Chroma with metadata indices for jurisdiction, case type, decision date, and source citation. Hybrid search combines semantic similarity with metadata filters (e.g., 'only retrieve cases from [target jurisdiction] after [relevant date]').

Stage 5: Retrieval & Re-Ranking

User case facts trigger a multi-pass retrieval: (1) semantic search for procedural and substantive relevance, (2) BM25 keyword search for statute/rule references, (3) cross-encoder re-ranking to identify top-5 most relevant authorities. Retrieved chunks are ranked by relevance score and source authority (e.g., Supreme Court > Circuit Court > District Court).

Stage 6: Prompt Context Assembly

Retrieved authorities are formatted into the LLM prompt as 'LEGAL CONTEXT' blocks, each tagged with source citation. The prompt explicitly instructs the model to reference only these retrieved sources and to cite every claim.

5. GENERATIVE AI & ADVERSARIAL PROMPTING STRATEGY

5.1 LLM Model Selection

The system uses Claude 3.5 Sonnet (or equivalent state-of-the-art model) for adversarial argument synthesis. This model is selected for: (1) superior reasoning and argument construction, (2) strong instruction-following for role-play constraints, (3) native support for streaming via SSE for timeout mitigation, (4) documented safety and hallucination resistance with grounded context.

5.2 Adversarial Prompting Framework

The system prompt is engineered to enforce strict adversarial role-play while constraining outputs to retrieved sources:

SYSTEM PROMPT TEMPLATE (Pseudocode):

"You are an experienced opposing counsel preparing a defense against a [CLAIM_TYPE] claim in [JURISDICTION] court. Your goal is to construct the strongest possible counter-arguments, procedural objections, and counterevidence using ONLY the legal authorities below.

LEGAL CONTEXT:

[Retrieved case law and statutes with citations]

LITIGANT'S CASE SUMMARY:

[Structured case facts]

Your response must:

1. Highlight 3-5 critical vulnerabilities in the plaintiff's case
2. Cite specific cases, statutes, and procedural rules supporting each defense
3. Identify procedural defects (e.g., notice failures, jurisdictional issues)
4. Suggest likely discovery disputes or evidentiary objections
5. Assign confidence scores (High/Medium/Low) to each argument
6. Include footnote citations to retrieved sources

Do NOT invent cases, statutes, or authorities. If you cannot support a point with the retrieved context, explicitly state so."

5.3 Output Formatting & Verification

- LLM outputs structured JSON with keys: counter_arguments[], procedural_defenses[], counterevidence[], citations[]
- Each counter-argument includes: claim_text, supporting_authority[], confidence_score
- Citations are parsed via regex to extract case names and statute references
- Citation Verification Engine cross-references extracted citations against retrieved document chunks

- Unverified citations are flagged and removed; response regenerated if verification fails catastrophically

5.4 Hallucination Prevention & Grounding Assurance

The Grounding Verification metric (G_v) rigorously tracks hallucination risk:

Grounding Verification Formula: $G_v = C_v / C_t = 1.00$

Where C_v = count of verified citations (matched to retrieved chunks via regex) and C_t = total citation count in output. Target: $G_v = 1.00$ (100% verification). Any response with $G_v < 0.95$ is flagged for manual review or regenerated.

6. VECTOR DATABASE & EMBEDDING STRATEGY

6.1 Embedding Model Selection

Primary (Recommended): sentence-transformers/all-MiniLM-L6-v2: 384-dimensional, fast (~100 docs/sec on CPU), strong legal domain performance via fine-tuning on legal corpora. Lightweight and suitable for Colab/serverless.

Alternative (Enhanced): Legal-BERT or domain-optimized models: 768-dimensional, trained on legal texts, superior jurisdiction-specific retrieval. Slower (~20 docs/sec) but higher precision for edge cases.

Fallback: OpenAI Text Embedding 3 (small): 1536-dimensional, API-based, production-grade quality. Higher latency and cost; use for production if Colab-based pipeline proves insufficient.

6.2 Vector Database: Qdrant vs. Chroma

Qdrant (Recommended for Production): • Distributed, scalable, supports hybrid search (dense + sparse vectors) • REST API for cloud deployment • Supports dynamic indexing and real-time updates • Ideal for 100k+ document collections • Free cloud tier sufficient for MVP

Chroma (Recommended for Development/Colab): • Lightweight, in-process or client-server • Easy integration with Langchain/LlamaIndex • Built-in metadata filtering • Sufficient for <50k documents • Perfect for initial prototyping and Colab notebooks

6.3 Metadata Schema & Filtering

Metadata attached to each embedded chunk:

```
{
  "case_name": "State v. Smith",
  "citation": "123 N.E.2d 456 (Ohio Ct. App. 2015)",
  "jurisdiction": "Ohio",
  "court_level": "appellate",
  "decision_date": "2015-03-15",
  "claim_types": ["criminal_defense", "evidence", "procedure"],
  "legal_topics": ["motion_to_suppress", "chain_of_custody"],
  "document_chunk_id": "case_123_chunk_5",
  "source_url": "caps.law.harvard.edu/..."
}
```

These metadata fields enable hybrid queries: semantic similarity + filtered constraints (e.g., 'find arguments for [claim_type] in [jurisdiction] after [date]').

7. IMPLEMENTATION ROADMAP (4 MODULAR STEPS)

The project follows an incremental, cumulative architecture ensuring continuous testing and delivery:

Step 1: Baseline Architecture & Form-Guided Validation MVP

Objective: Establish foundational web framework and Vercel hosting baseline with mock responses

- Frontend: Clean Next.js (TypeScript) single-page app with multi-step form wizard
- Backend: FastAPI serverless route accepting case structures and returning mock JSON
- Deployment: GitHub + Vercel continuous deployment pipeline
- Deliverable: Live web application with functional case intake form

Step 2: Google Colab Legal Ingestion & Vector Layer

Objective: Integrate real legal database and operational vector database index

- Colab Notebook: Stream Harvard CAP dataset from Hugging Face (free-law/Caselaw_Access_Project)
- Processing: Generate embeddings using all-MiniLM-L6-v2; store in Chroma or Qdrant Cloud
- API Upgrade: Replace mock endpoints with live vector-store queries returning relevant source texts
- Deliverable: Backend API returning real legal authorities matching case queries

Step 3: Adversarial Simulation Engine & SSE Streaming

Objective: Implement generative AI with adversarial prompting and serverless timeout mitigation

- Adversarial Prompt: Design role-play constraints for opposing counsel argument synthesis
- LLM Integration: Connect to Claude API for grounded argument generation
- SSE Streaming: Implement Serverless Event Streaming via Python generators to bypass 15s timeout
- Frontend Update: Next.js streaming UI rendering tokens as they arrive from backend
- Deliverable: Real-time streaming adversarial arguments grounded in retrieved case law

Step 4: Rebuttal Interface, Verification, & PDF Export

Objective: Deliver final refined application with legal safeguards and comprehensive tools

- Rebuttal UI: Split-panel interface with simulated arguments (left) and rebuttal text boxes (right)

- Verification Framework: Regex-based citation validation ensuring all claims map to retrieved sources
- PDF Export: Client-side 'Download Hearing Rehearsal Guide' combining case facts, simulations, rebuttals
- Legal Disclaimers: Non-dismissible overlay stating educational use only
- Deliverable: Production-ready hearing rehearsal application with complete audit trail

8. DEPLOYMENT & PRODUCTION ARCHITECTURE

8.1 Vercel Serverless Functions & Timeout Mitigation

Standard Vercel hobby-tier serverless functions impose a 15-second execution timeout. RAG operations—particularly vector similarity search over 100k+ documents followed by LLM token generation—frequently exceed this threshold. The solution is **Serverless Event Streaming (SSE)**:

Timeout Mitigation Strategy (SSE Pattern):

1. User submits case facts to `/api/generate-opposition`
2. Backend immediately returns a streaming response (HTTP 200 with `Transfer-Encoding: chunked`)
3. Vector search and LLM generation proceed on the open stream
4. Tokens are written to the stream instantly as generated (e.g., every 100ms)
5. Each token write resets the connection timeout clock
6. Frontend continuously reads stream buffer, rendering text blocks as they arrive
7. Total response latency: ~3-5 seconds, well below 15s timeout

This pattern allows LLM completions of 2000+ tokens within serverless constraints.

8.2 Backend Architecture (FastAPI)

Python FastAPI Application Structure:

```
api/  
  ├── main.py (FastAPI app initialization, routes)  
  ├── routes/  
  │   ├── intake.py (POST /api/intake)  
  │   ├── retrieve.py (POST /api/retrieve)  
  │   └── generate_opposition.py (POST /api/generate-opposition with SSE)  
  ├── services/  
  │   ├── case_parser.py (Extract structured JSON from free-text)  
  │   ├── retrieval_service.py (Query vector DB, rank results)  
  │   ├── llm_service.py (Claude API integration with streaming)  
  │   └── citation_verifier.py (Regex-based hallucination detection)  
  ├── models/  
  │   ├── case_schema.py (Pydantic models for validation)  
  │   └── responses.py (Structured output schemas)  
  └── config/  
      ├── qdrant_config.py (Vector DB credentials)  
      ├── anthropic_config.py (Claude API key management)  
      └── constants.py (System prompts, thresholds)
```

8.3 Frontend Architecture (Next.js TypeScript)

Next.js Application Structure:

```
src/  
  ├── pages/  
  │   ├── index.tsx (Landing page)  
  │   ├── intake.tsx (Multi-step case intake form)  
  │   └── simulation.tsx (Opposing arguments display + rebuttal interface)
```

```

■   ■■■ api/
■       ■■■ [route-name].ts (API routes forwarding to FastAPI)
■■■ components/
■   ■■■ CaseIntakeForm.tsx (Form wizard with validation)
■   ■■■ StreamingArgumentDisplay.tsx (Renders streaming LLM text)
■   ■■■ RebuttalPanel.tsx (User rebuttal text entry)
■   ■■■ DisclaimerOverlay.tsx (Non-dismissible legal notice)
■■■ services/
■   ■■■ api.ts (HTTP client with streaming support)
■   ■■■ pdfExport.ts (Generate PDF from case + simulation data)
■■■ styles/
■   ■■■ globals.css (Tailwind CSS, responsive design)
■■■ types/
    ■■■ index.ts (TypeScript interfaces for case, arguments, etc.)

```

8.4 Deployment Workflow (GitHub → Vercel)

- Push code to GitHub repository with protected main branch
- Vercel automatically detects commits and triggers preview/production builds
- Environment variables (Qdrant API key, Claude API key) configured in Vercel dashboard
- Database migrations for vector indices run in separate Colab pipeline, output published to Qdrant Cloud
- FastAPI backend deployed as Vercel Serverless Functions (one function per route)
- Next.js frontend deployed to Vercel Edge Network (auto-minified, cached)
- Domain configured (e.g., opposing-argument-simulator.vercel.app)
- Continuous monitoring via Vercel Analytics and error tracking (Sentry integration)

9. EVALUATION METRICS & QUALITY ASSURANCE

9.1 Citation Accuracy Evaluation (Hallucination Detection)

Grounding Verification (G_v)

Primary metric measuring hallucination risk. For each generated response:

- Extract all case names and statute citations from LLM output via regex
- Cross-reference against retrieved document metadata and chunk text
- Count verified citations (C_v) vs. total citations (C_t)
- $G_v = C_v / C_t$ — target ≥ 0.95 (95% verification minimum)
- Log unverified citations for manual review (potential hallucinations)

RAGAS Framework Integration

Implement RAGAS (Retrieval-Augmented Generation Assessment) metrics to evaluate retrieval and generation quality independently:

- **Faithfulness:** Does generated argument align with retrieved legal sources? (Target: >0.80)
- **Answer Relevance:** Does output directly address case facts? (Target: >0.85)
- **Context Relevance:** Are retrieved documents actually pertinent? (Target: >0.75)
- **Context Precision:** What fraction of retrieved contexts are relevant? (Target: >0.70)

9.2 Retrieval Performance Metrics

Mean Reciprocal Rank (MRR): Position of first relevant document in retrieval ranking. Target: MRR >0.70

Normalized Discounted Cumulative Gain (NDCG): Ranking quality accounting for relevance grades. Target: NDCG@5 >0.65

Latency: Time from case query to top-k retrieval. Target: $<500\text{ms}$ for 100k document corpus

Recall: Proportion of all relevant documents retrieved. Target: Recall@20 >0.60

9.3 Generation Quality & Latency

BLEU / ROUGE Scores: Measure textual overlap with gold-standard opposing arguments drafted by legal experts. (Reference-based evaluation)

Token Latency: Time to first token (TTFT): <300ms. Average token latency: <50ms. Total streaming latency: <3.0s for 500-token response.

Output Length: Target: 400-800 tokens per opposing argument (sufficient for comprehensive but focused rebuttal preparation).

Argument Diversity: Number of distinct counter-argument categories identified. Target: ≥ 3 per case.

9.4 User-Level Evaluation (A/B Testing)

- Recruit self-represented litigants preparing for real hearings
- Randomize: Control group (no simulator) vs. Treatment group (with simulator)
- Measure outcomes: Hearing preparedness self-assessment, argument anticipation accuracy, perceived confidence
- Secondary outcome: Post-hearing survey on argument quality and decision impact
- Target: Treatment group shows statistically significant improvement in preparedness ($p < 0.05$)

10. SAFETY FRAMEWORKS & LEGAL SAFEGUARDS

10.1 Hallucination Prevention & Verification Layers

Layer 1: Prompt Constraint: System prompt explicitly forbids citation of non-retrieved authorities. LLM is instructed: 'If you cannot find supporting authority in the LEGAL CONTEXT, explicitly state so.'

Layer 2: Citation Extraction & Verification: All citations extracted via deterministic regex matching. Each citation string cross-referenced against retrieved document metadata and text content.

Layer 3: Grounding Validation (G_v): Every response scored on Grounding Verification metric. Responses with $G_v < 0.95$ are flagged for regeneration or manual review.

Layer 4: Output Filtering: Automatic removal of any unverified citations before user presentation. Fallback: regenerate response with lower k (fewer retrieved contexts) to simplify reasoning.

Layer 5: Human Review Audit Trail: All low-confidence responses (< 0.80 Grounding Verification) logged for periodic human legal expert review. Findings inform prompt refinement.

10.2 Disclaimer & User Framing

MANDATORY DISCLAIMER (Non-Dismissible Overlay):

"IMPORTANT LEGAL NOTICE: This tool is NOT a substitute for qualified attorney representation. It provides educational simulation for case preparation practice only. The opposing arguments generated here are for training and preparedness purposes only and do NOT constitute legal advice or a comprehensive legal analysis.

Self-represented litigants should:

- Consult with a licensed attorney if possible
- Verify all legal information independently
- Not rely solely on this tool for court strategy
- Understand that actual opposing counsel may raise different or stronger arguments

The system uses artificial intelligence to retrieve and synthesize legal authorities. While efforts are made to ensure accuracy, no guarantee of legal correctness is made."

This notice appears:

- On page load (before case entry)
- Above every simulated opposing argument (side-by-side)
- In the PDF export (header + footer on each page)
- In the Terms of Service (mandatory acceptance before use)

10.3 Data Privacy & Minimization

- **No Persistent Storage:** Case details remain in browser session or transient memory only. No server-side database of user cases.
- **Encryption in Transit:** All data transmitted via HTTPS with TLS 1.3. API calls to Claude and Qdrant encrypted end-to-end.
- **No Analytics Tracking:** Specific case content is never logged, sampled, or used for product analytics. Aggregate metrics only (e.g., 'Query latency: 450ms').
- **User Consent:** Privacy policy and data handling practices clearly disclosed before user account creation.
- **Right to Deletion:** Users can request deletion of session data at any time (implemented via session cache purge).

11. TECHNICAL SPECIFICATION & CODE PATTERNS

11.1 Data Ingestion Pipeline (Google Colab)

```
# Harvard CAP Dataset Streaming in Colab

from datasets import load_dataset
from langchain.text_splitter import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
import qdrant_client

# 1. Stream CAP dataset from Hugging Face (avoids OOM)
dataset = load_dataset(
    "free-law/Caselaw_Access_Project",
    split="train",
    streaming=True # Critical: prevents loading entire dataset into RAM
)

# 2. Initialize embedding model (lightweight, fast)
model = SentenceTransformer('all-MiniLM-L6-v2') # 384-dim, ~10MB

# 3. Streaming ingestion loop
client = qdrant_client.QdrantClient(":memory:") # Local for testing
collection_name = "legal_opinions"

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=400,
    chunk_overlap=50,
    separators=["\n\n", "\n", " ", ""]
)

for idx, case_record in enumerate(dataset.take(50000)): # Stream 50k cases
    # Extract metadata
    case_name = case_record.get('name', 'Unknown')
    citation = case_record.get('citation', '')
    decision_date = case_record.get('decision_date', '')
    opinion_text = case_record.get('opinion', '')

    # Split into chunks
    chunks = text_splitter.split_text(opinion_text)

    # Generate embeddings and store
    for chunk_idx, chunk in enumerate(chunks):
        embedding = model.encode(chunk)

        client.upsert(
            collection_name=collection_name,
            points=[{
                "id": idx * 1000 + chunk_idx,
                "vector": embedding.tolist(),
                "payload": {
                    "case_name": case_name,
                    "citation": citation,
                    "decision_date": decision_date,
```

```

        "chunk_text": chunk,
        "chunk_index": chunk_idx
    }
}
    })
)

if (idx + 1) % 1000 == 0:
    print(f"Processed {idx + 1} cases...")

print("Ingestion complete!")

```

11.2 FastAPI Endpoint: RAG-Grounded Generation with SSE

```

# FastAPI route with Serverless Event Streaming (SSE)

from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
import asyncio
from anthropic import Anthropic

app = FastAPI()
client = Anthropic()

@app.post("/api/generate-opposition")
async def generate_opposition(case_data: dict):
    """
    Generate opposing arguments via RAG-grounded LLM
    Streams response to bypass Vercel 15s timeout
    """

    # 1. Extract case facts
    jurisdiction = case_data.get('jurisdiction')
    claim_type = case_data.get('claim_type')
    case_facts = case_data.get('case_facts')

    # 2. Retrieve relevant legal authorities via vector search
    query_embedding = embed_model.encode(case_facts)
    retrieved_cases = qdrant_client.search(
        collection_name="legal_opinions",
        query_vector=query_embedding.tolist(),
        limit=10, # Top-10 most relevant
        score_threshold=0.60
    )

    # 3. Format legal context for prompt
    legal_context = "\n\n".join([
        f"Case: {case['payload']['case_name']}\n",
        f"Citation: {case['payload']['citation']}\n",
        f"Text: {case['payload']['chunk_text']}"
        for case in retrieved_cases
    ])

    # 4. Build system prompt
    system_prompt = f"""You are experienced opposing counsel in a {claim_type}
case in {jurisdiction} jurisdiction. Generate the strongest counter-arguments

```

using ONLY the legal authorities provided below.

LEGAL CONTEXT:
{legal_context}

Respond with 3-5 key counter-arguments, each with:

- Claim description
- Supporting case citation(s)
- Confidence level (High/Medium/Low)

Do NOT invent cases or statutes. Only cite retrieved authorities."""

```
# 5. Stream response from Claude API
async def stream_response():
    with client.messages.stream(
        model="claude-3-5-sonnet-20241022",
        max_tokens=1000,
        messages=[{"role": "user", "content": case_facts}],
        system=system_prompt
    ) as stream:
        for text in stream.text_stream:
            yield f"data: {json.dumps({'delta': {'text': text}})}\n\n"
            await asyncio.sleep(0) # Yield control to event loop

    return StreamingResponse(
        stream_response(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no"
        }
    )
```

11.3 Next.js Streaming Frontend

```
// React component reading SSE stream

import { useState } from 'react';

export default function SimulationPanel() {
    const [opposingArguments, setOpposingArguments] = useState('');
    const [isStreaming, setIsStreaming] = useState(false);

    const handleGenerateOpposition = async (caseData) => {
        setIsStreaming(true);
        setOpposingArguments('');

        try {
            const response = await fetch('/api/generate-opposition', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(caseData)
            });

            if (!response.ok) throw new Error('API error');
```

```

const reader = response.body.getReader();
const decoder = new TextDecoder();

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  const text = decoder.decode(value);
  const lines = text.split('\n');

  for (const line of lines) {
    if (line.startsWith('data: ')) {
      const data = JSON.parse(line.slice(6));
      setOpposingArguments(prev =>
        prev + (data.delta?.text || '')
      );
    }
  }
} finally {
  setIsStreaming(false);
}
};

return (
  <div className="simulation-panel">
    <h2>Opposing Arguments (Simulated)</h2>
    <div className="streaming-text">
      {opposingArguments}
      {isStreaming && <span className="cursor">|</span>}
    </div>
    <div className="rebuttal-panel">
      <textarea
        placeholder="Enter your counter-argument..."
        disabled={isStreaming}
      />
    </div>
  </div>
);
}

```

12. APPENDICES

12.1 Glossary of Key Terms

RAG (Retrieval-Augmented Generation): Technique combining document retrieval with LLM generation. Retrieved documents ground the model's response, reducing hallucinations.

Vector Database: Database storing high-dimensional embeddings and enabling semantic similarity search.

Embedding: Dense numerical representation (e.g., 384-dimensional vector) of text capturing semantic meaning.

Hallucination: LLM generating plausible but false information, e.g., fabricated case citations.

SSE (Serverless Event Streaming): HTTP streaming technique allowing continuous data transfer without request timeout.

Grounding Verification (G_v): Metric measuring fraction of LLM citations verified against retrieved sources.

Self-Represented Litigant (SRL): Individual appearing in court without attorney representation (pro se).

Jurisdiction-Grounded: Retrieval and generation constrained to legal authorities from specific geographic or judicial jurisdiction.

12.2 Technology Stack Summary

Frontend: Next.js (TypeScript), Tailwind CSS, React Hooks

Backend: FastAPI (Python), Anthropic SDK, Langchain/LlamaIndex

Vector Database: Qdrant Cloud (production) / Chroma (development)

Embeddings: sentence-transformers all-MiniLM-L6-v2

Legal Data Source: Harvard CAP (Hugging Face: free-law/Caselaw_Access_Project)

LLM: Claude 3.5 Sonnet (Anthropic API)

Deployment: Vercel (serverless functions + static hosting)

CI/CD: GitHub Actions, Vercel continuous deployment

Data Processing: Google Colab (free, GPU-enabled for embedding generation)

PDF Generation: Reportlab (Python) / puppeteer (Node.js)

12.3 Project Timeline & Milestones

- M1 (Week 1-2): Frontend setup, form validation, Vercel baseline deployment
- M2 (Week 3-4): CAP dataset ingestion, embedding generation, Qdrant integration
- M3 (Week 5-6): Claude API integration, adversarial prompting, SSE streaming implementation
- M4 (Week 7-8): Rebuttal interface, citation verification, PDF export, comprehensive testing
- M5 (Week 9-10): User acceptance testing, metric evaluation, documentation
- M6 (Week 11-12): Production launch, monitoring setup, initial user feedback collection

12.4 Future Enhancements & Roadmap

- **Multi-Jurisdiction Support:** Expand beyond initial target jurisdiction to all 50 US states + federal courts
- **Real-Time Case Updates:** Automated ingestion of newly decided cases (daily/weekly refresh)
- **Specialized Domain Models:** Fine-tune embedding models on legal text for improved retrieval accuracy
- **Interactive Cross-Examination Simulation:** Beyond static opposing arguments, simulate live oral argument scenarios
- **Video Tutoring Integration:** Pair arguments with expert video commentary on counter-strategies
- **Mobile App:** iOS/Android native app for case rehearsal on mobile devices
- **Integration with Court Dockets:** Auto-populate case facts from public court records (PACER, state court APIs)
- **Predictive Outcome Modeling:** Estimate hearing outcome probability based on case similarity to historical decisions

12.5 References & Resource Links

Harvard Caselaw Access Project: <https://case.law/>

Hugging Face CAP Dataset: https://huggingface.co/datasets/free-law/Caselaw_Access_Project

Langchain Documentation: <https://python.langchain.com/>

LlamaIndex (formerly GPT Index): <https://www.llamaindex.ai/>

Qdrant Vector DB: <https://qdrant.tech/>

Chroma Vector DB: <https://www.trychroma.com/>

Anthropic Claude API: <https://www.anthropic.com/>

Vercel Deployment Docs: <https://vercel.com/docs>

RAGAS Evaluation Framework: <https://github.com/explodinggradients/ragas>

Sentence Transformers: <https://www.sbert.net/>

FastAPI Documentation: <https://fastapi.tiangolo.com/>

Next.js Documentation: <https://nextjs.org/docs>

CLOSING STATEMENT

This Opposing-Argument Simulator represents a transformative application of retrieval-augmented generation and adversarial AI to the critical challenge of access-to-justice. By combining the latest advances in generative AI with rigorous legal grounding and sophisticated safety verification, this system shifts the preparatory burden from professional counsel to AI—democratizing the ability of self-represented litigants to anticipate and rebut opposing arguments before stepping into the courtroom.

The technical architecture is production-ready, the RAG pipeline is hallucination-resistant, and the deployment strategy leverages serverless scalability and streaming optimization to meet real-world performance demands. The project is organized into four modular, testable implementation steps enabling rapid iteration and continuous validation against the highest standards of legal accuracy and user safety.

With this comprehensive SRS and technical architecture as the foundation, the Opposing-Argument Simulator is positioned to become an essential tool for equitable civil justice practice.

Hasana Zahid
Bachelor of Science in Artificial Intelligence
COMSATS University Islamabad
July 2026